Black Bear Foodshare

Code Inspection Report

Olive Food Solutions

Dr. Scott Marzilli

Team Members: Dumas Wesley, Kaulenas Corey, Moody-Broen Makai, Sima Denis, Sholler Jakob

Table of Contents

Page

1. Introduction.....X
   1.1 Purpose of This Document..... X
   References.....X
   1.2 Coding and Commenting Conventions.....X
   1.3 Defect Checklist.....X
2. Code Inspection Process.....X
   2.1 Description.....X
   2.2 Impressions of the Process.....X
   2.3 Inspection Meetings.....X
3. Modules Inspected.....X
4. Defects.....X

Appendix A - Coding and Commenting Conventions.....X
Appendix B – Team Review Sign-off.....X
Appendix C – Document Contributions..... X

1. Introduction

Jack of all Spades is designing software to help students use the UV-vis spectrometer and IR spectrophotometer. Since they will be used in introductory labs, many students will need to use these tools. The software should take the data from a sample and upload it to InterChemNet (ICN), allowing the student to analyze the data later, which is the part that normally takes the most time. It should also use presets to make it more user friendly for students new to chemistry. The software will help all the students in the labs to complete their labs in a timely manner.

1.1 Purpose of This Document

The purpose of this document is to provide a comprehensive and thorough overview of Jack of All Spades completed code thus far. This will involve a review of coding conventions, commenting conventions, and code review practices, compiling of remaining issues, and post review introspection. The intended viewership of this document is the Jack of All Spades team, clients, and instructors.

References

- Jack of All Spades, System Requirements Specification
  https://github.com/Eric-Jestel/COS-397-Black-2025/blob/main/Signed SRS V1.1 (MB).pdf
- Jack of All Spades, Software Design Document
  https://github.com/Eric-Jestel/COS-397-Black-2025/blob/main/Signed SDD v1.0 (MB).pdf

1.2 Coding and Commenting Conventions

The Jack of All Spades team has implemented industry standards typical for Agile development. They conduct development through assigned modules which together create entire sprint objectives. For instance, front end development was broken into functional modules and developed in isolated branches by assigned team members. Upon completion code was merged from a Pull Request to be self and peer reviewed. Branches were often deleted upon merging, this makes the history of the project more difficult to examine.

Commenting conventions are designed to allow for easily navigable code which should be understandable for all team members, clients, and third parties. All frontend structures should have explanatory comments providing a name and purpose to the included function. All docstrings should be formatted using Google's style, enforced by flake8 and black.

Backend contains comments with the intention of explaining functionality for individual functions, providing branch explanations, variable naming conventions, intended functionality, etc. If there are sections which contain errors or unfinished functions these are highlighted with comments pertaining to the objective.

Overall, the backend uses a Three Controller Model, while the frontend uses a Model-ViewModel architecture. Code is automatically formatted using flake8 and black with a max line length of 88. The docstrings are in Google's format.

1.3 Defect Checklist

Name	**Category**
Inconsistent Names	Coding Convention
Dead Code	Coding Convention
Missing Docstrings	Commenting
Hardcoded Elements	Logic Error
Bypass of Design	Logic Error
Missing Input Validation	Security Issue
Inadequate Error Handling	Logic Error
Magic Numbers	Coding Convention
Missing Class Documentation	Commenting
CICD Failure	DevOps
Verification of User Identity	Security Issue
Unclear Module Name	Coding Convention
Confusing Function Identity	Design
Formatting Failure	DevOps
Missing Type Hints	Coding Convention
Unused Imports	Coding Convention
No Context Manager for Files (with …)	Coding Convention
Inconsistent Error Handling Pattern	Logic Error
Resource Leak	Security Oversight
Public Method Should Be Private	Security Oversight

Loose Files	Coding Convention

2. Code Inspection Process

The code inspection was conducted as a two stage group effort. The preliminary inspection was conducted by the Olive Food Solutions team as a private group inspection. We combed through the project in segmented modules, first analyzing the frontend to develop an idea of the project's scope and functionality from a user perspective. We then delved into the backend of this project, which is considerably more complex given the nature of the Jack of All Spades project.

2.1 Description

The process followed by the Olive Food Solutions team for the code review of Jack of All Spades began with a preliminary reading, conducted individually by all team members in attendance. This undertaking was carried out with deliberate intention according to our individual strengths. Corey and Wesley, our backend developers, focused on the aspects of the Jack of All Spade's backend as they had the strongest understanding of the conventions and practices. Jacob and Makai spent more time analyzing front end code, documentation, project structure, and clarity. After all attending members had time to read through the Jack of All Spades code, a group meeting was conducted, whereupon the Olive Food Solutions team read through the Jack of All Spades code together, discussing the individual defects of style, clarity, code and structure. Once the Oliver Food Solutions team finished discussing the discovered defects, a document was compiled to prepare what we wanted to discuss during the code review with Jack of All Spades.

The code review was ultimately a productive exchange of critical feedback and explanatory retrospection. The Jack of All Spades team provided well thought out explanations for the aforementioned defects. Many issues we found were intrinsically tied to the nature of the Jack of All Spades project, given that they are migrating proprietary software with little access to the necessary testing equipment and no source code. The situational circumstances caused some hiccups throughout the code review, as the Olive Food Solutions team was not able to fully recognize the function and design of aspects of the Jack of All Sades code. As a unit, both teams were able to analyze these issues and brainstorm solutions which will hopefully prove to be beneficial for the future of the Jack of All Spades project.

2.2 Impressions of the Process

Overall, the code inspection was a beneficial process. It allowed us to be a second set of eyes to review another team's code and check for any possible errors that the original developers

may have missed. Looking over their code from an outside perspective helped us notice potential issues that could have been hard to see for the developers who are familiar with their code, who might have just glossed over the issues without noticing them. In this regard, the code inspection was a very useful process for catching stray errors, and ensuring that the other team's code was easy to read and understand. One of the problems with the code review was that our team only had access to an old copy of the System Requirements Specification, so we were unable to compare their code against the most recent version of their requirements, which may have changed since we received the original System Requirements Specification.

The prototype app modular unit is the least likely to have remaining flaws, as we did not find very many errors present in the code while reading through it, and it appears to have been well tested, ensuring that it is working properly. The prototype app could still benefit from having more comments, as it is currently lacking comments for the majority of the functionality that it provides, and having more comments would aid in its readability and ability to find errors. Overall the prototype app modular unit appears to be relatively bug free, but would benefit from more documentation.

The server controller is one of the modular units that is most likely to have remaining flaws, as it currently lacks error handling for file operations, has hard coded API URLs, and is missing numerous comments. These are the problems that we spotted while reading over the code, but it is possible that we missed some in our look through. Overall, the server controller could benefit from having more comments explaining what everything does, and while making those comments, ensuring that everything has proper error handling, so that there are no unforeseen problems that arise, it would also benefit from using variables to remove any hard coded urls to prevent errors and make editing it easier in the future.

2.3 Inspection Meetings

An inspection meeting was held on Wednesday, March 4th from 2:40pm-3:15pm in person.
The roles were as follows:

- Corey Aras Kaulenas: Moderator
- Jakob Sholler: Recorder
- Makai Moody-Broen: Inspector
- Wesley Dumas: Inspector
- Jack of all Trades: Author

Code Units Covered:

- PrototypeApp
- InstrumentController
- ServerController
- SystemController

3. Modules Inspected

This contains a list of all the modules used in the code. For each module, the name, description of the functionality, and architecture function are provided. Also, for every module, the changes between the System Design Document (SDD) and the code are mentioned. All modules in the SDD are completed or close to complete.

Name	**InstrumentController**
Functionality	Controls the instrument present on the lab computer, sending signals and gathering data from the hardware. Converts the instrument signals into understandable commands for the SystemController. Captures samples, adjusts instrument settings, and formats instrument output into csv.
Architecture Function	Sits between the SystemController and the instrument, making sure programmatic commands from the SystemController can be interpreted by the instrument and vice versa.
Changes from SDD	N/A

Name	**ServerController**
Functionality	Formats data into proper API calls for the ICN Server. Handles logins and API requests
Architecture Function	Between the SystemController and ICN Server, where the SystemController calls the ServerController when it wants to interact with the ICN Server.
Changes from SDD	Renamed from ICN Controller to ServerController

Name	**SystemController**
Functionality	Runs the main program loop, and handles high level calls to the Server and Instrument Controllers. Also checks the User Interface for interaction.
Architecture Function	Main controller from the system. Receives commands from the User Interface and sends appropriate calls to the Server/Instrument Controller.
Changes from SDD	Renamed from Program Controller to SystemController

Name	**AppPrototype**
Functionality	Handles rendering for the UI, and collects commands from the user.
Architecture Function	Renders the state of the app to the user. Collects commands from the user and sends them to the SystemController.
Changes from SDD	None, but it is currently named AppPrototype instead of User Interface.

4. Defects

This section contains a table containing the defects found within the code, including the Module, Description, Category, and Location. Most of these are definite defects, but some are noted as recommendations, or otherwise may be irrelevant to the code.

Name of Module	Description	Category	Location
SystemController	In runLabMachine() method, there's a hardcoded if 1: condition that bypasses server connectivity check	Correctness	components/SystemController.py lines 157-159
ServerController	Uses hardcoded API URL instead of configurable URL	User friendliness	components/ServerController.py line 40
ServerController	Missing error handling for file operations in parse_csv	Correctness	components/ServerController.py lines 154-162
All components	Missing many docstrings for private methods. Not necessarily a defect if intentional	Commenting convention	All files
SystemController	Public method takeSample doesn't have a docstring	Commenting convention	components/SystemController.py lines 261
SystemController	Public method	Commenting	components/Instrume

	changeSettings doesn't have a docstring	convention	ntController.py line 348
ServerController	Public method validate has an incomplete docstring that doesn't explain what it does	Commenting convention	components/ServerController.py line 175
InstrumentController	Uses hardcoded Windows path for ADL file instead of configurable path	User Friendliness	components/InstrumentController.py line 74-76
All files	Inconsistent module and function naming scheme	Correctness	All files
ServerController	Missing proper validation of CSV parsing results in parse_csv() method	Correctness	components/ServerController.py lines 154-162
InstrumentController	Class design could benefit from better encapsulation - many public attributes and methods that should be private or properly structured	Design	components/InstrumentController.py
SystemController	Use constants or something instead of hard coded error values	Design	components/SystemController.py
PrototypeApp	Inconsistent handling of state. Some state is in state.py, some are members of App	Design/Correctness	components/UserInterface/app/app.py and components/UserInterface/app/state.py
DevOps	Flake8 uses two different line lengths, 88 in the pre-commit, and 120 in .flake8	Correctness	.flake and .pre-commit-config.yaml

Overall	No comprehensive class-level docstrings for major classes (SystemController, InstrumentController, ServerController)	Commenting convention	All files
InstrumentController	Missing input validation in methods like take_blank, set_blank, no checking if filename parameters are valid	Correctness	components/InstrumentController.py lines 241, 279
DevOps	Tests are not required to pass for devops to succeed	Correctness	.pre-commit-config.yaml
AppPrototype	Default UIState has server status “OK” which could lead to bugs	Correctness	components/UserInterface/app/state.py line 11
Readme	Is not comprehensive, needs more information on how to run app, linters, tests	Other	README.md
File Tree	This is either working and should replace whatever it is replacing, or not and should be deleted	Coding Convention	components/pleasework.py

Appendix A – Team Review Sign-off

All team members have thoroughly reviewed this document and reached full agreement on all of its content. No major concerns have been raised, and any minor concerns or clarifications raised by individuals are noted in the comments below their signature. Additionally, all members of the team have agreed upon the formatting and presentation of this document with no major complaints.

Wesley Dumas
Signature: Wesley Dumas Digitally signed by Wesley Dumas Date: 2026.03.11 19:29:58 -04'00' Date: 03/11/2026
Comments:

Corey Kaulenas
Signature: Corey Kaulenas Digitally signed by Corey Kaulenas Date: 2026.03.11 20:00:45 -04'00' Date: 03/11/2026
Comments:

Makai Moody-Broen
Signature: Makai Moody-Broen Digitally signed by Makai Moody-Broen Date: 2026.03.11 19:56:51 -04'00' Date: 03/11/2026
Comments:

Denis Sima
Signature: Denis Sima Digitally signed by Denis Sima Date: 2026.03.11 19:26:56 -04'00' Date: 03/11/2026
Comments:

Jakob Sholler
Signature: Jakob Sholler Digitally signed by Jakob Sholler Date: 2026.03.11 19:37:04 -04'00' Date: 03/11/2026
Comments:

Appendix B – Document Contributions

Sholler Jakob, Contributions:

Dumas Wesley, Contributions:

- Part of the Defect Checklist
- Impressions of the Process

Moody-Broen Makai, Contributions:

- Introduction
- Purpose
- Coding and Commenting Conventions
- Part of defect list
- Code inspection process
- Description

Kaulenas Corey, Contributions:

- Defect Checklist
- Defect Section
- Coding and Commenting Conventions
- Modules Inspected

Sima Denis, Contributions: